

POP Topic 5 Final Report

Brygida Silawko
Tomasz Smoleń

October 7, 2026

1 Problem description

The problem concerns reducing excessively fast convergence in evolutionary algorithms, which results in getting stuck in local optima. Evolutionary algorithms have various mechanisms responsible for exploration (e.g. mutation) and exploitation (selection, crossover). In the end, an algorithm will often either exploit too much (and therefore easily get stuck in local optima) or explore too much (and therefore run for a very long time). A solution to this problem is to add a heuristic that controls the current behaviour of the algorithm. We focus on the DG heuristic. The DG (Diversity Guided) heuristic uses a two-phase mechanism: Exploration, during which the population is mutated, and Exploitation, during which crossover and selection take place. To control which phase we are currently in, we use the ‘diversity’ measure, computed as follows:

$$diversity(P) = \frac{1}{|L| \cdot |P|} \cdot \sum_{i=1}^{|P|} \sqrt{\sum_{j=1}^N (s_{ij} - \bar{s}_j)^2} \quad (1)$$

$|L|$ - length of the diagonal of the search space $S \subseteq \mathbb{R}^N$

P - population, $|P|$ - population size

N - problem dimensionality

s_{ij} - j -th element of the i -th individual

$\bar{s}_j$ - j -th coordinate of the mean point $\bar{s}$

In equation (1) we normalize the sum over i by dividing by the length of the search space diagonal (the higher the problem dimensionality, the larger the sum over j).

Depending on whether our ‘diversity’ is greater or smaller than the values d_{min} and d_{high} , we change the phase we are in. As a result, the algorithm explores when the diversity of the population decreases, and exploits when the diversity is too high.

2 Numerical experiments

We carried out two main experiments:

2.1 ECDF curves based on the BBOB benchmark

We used the BBOB benchmark from the COCO framework to study and compare our algorithm with the implementation of differential evolution in the rand/1/bin variant from the SciPy library. We ran the benchmark for all functions in dimensions 2, 3, 5, 10, 20 and 40.

2.2 Convergence on the sphere function

Again we used the COCO framework, this time to compare the convergence of our algorithm and the SciPy implementation of DE/rand/1/bin. We performed this experiment for the sphere function (function f1 in COCO) in dimension 5.

3 Technology choice

The project was implemented in Python, using libraries such as numpy. In addition, for the experiments we use the coco-experiment and cocopp libraries. We used uv as the package manager.

4 Pseudocode of the modified DE/rand/1/bin

Compared with the pseudocode from the initial documentation, the way points are selected for crossover and mutation, and the way they are evaluated, has changed. Previously the point p_i took part in crossover and mutation; now it is used only in selection, where it is compared with the new individual produced by crossover or mutation. This approach is closer to classic DE, where the i -th point of the population (p_i) is compared with a new individual created from other randomly drawn working points. Thanks to these changes, our algorithm achieved a higher number of successes in the BBOB benchmark.

Algorithm 1 DE/rand/1/bin with DG

```

Initialize the initial population  $P_0$  of size  $\mu$ 
 $t \leftarrow 0$ 
mode  $\leftarrow$  Exploit
while !stop do
     $P_r \leftarrow P_t$   $\triangleright$  Working population representing the state of  $P_t$  after changes to
    individuals 1 to  $i$ 
    for  $i \in 1 : \mu$  do
        if  $diversity(P_r) < d_{low}$  then
            mode  $\leftarrow$  Explore
        else if  $diversity(P_r) > d_{high}$  then
            mode  $\leftarrow$  Exploit
        end if
         $p_i = P_t[i]$ 
        if mode == Exploit then
             $c_1, c_2 \leftarrow sample(P_t \setminus P_t[i], 2)$ 
            for  $j \in 1 : n$  do  $\triangleright n$  - length of an individual
                if  $rand(0, 1) < p_c$  then  $\triangleright p_c$  - crossover probability
                     $z_{i,j} \leftarrow c_{1,j}$ 
                else
                     $z_{i,j} \leftarrow c_{2,j}$ 
                end if
            end for
            if  $fitness(z_i) > fitness(p_i)$  then  $\triangleright$  or the opposite in case of minimization
                 $P_r[i] \leftarrow z_i$   $\triangleright$  Replace the  $i$ -th individual with the crossover result
            end if
        else  $\triangleright$  mode == Explore
             $x_1, x_2, x_3 \leftarrow sample(P_t \setminus P_t[i], 3)$ 
             $m_i \leftarrow x_3 + F(x_1, x_2)$ 
            if  $fitness(m_i) > fitness(p_i)$  then  $\triangleright$  or the opposite in case of minimization
                 $P_r[i] \leftarrow m_i$   $\triangleright$  Replace the  $i$ -th individual with the mutation result
            end if
        end if
    end for
     $P_{t+1} \leftarrow P_r$ 
     $t \leftarrow t + 1$ 
end while
Select the best individual from  $P_t$ 

```

5 Experimental results

5.1 BBOB benchmark

The results of the BBOB benchmark are as follows:

- de scipy - the SciPy implementation of DE
- de dg of - our algorithm

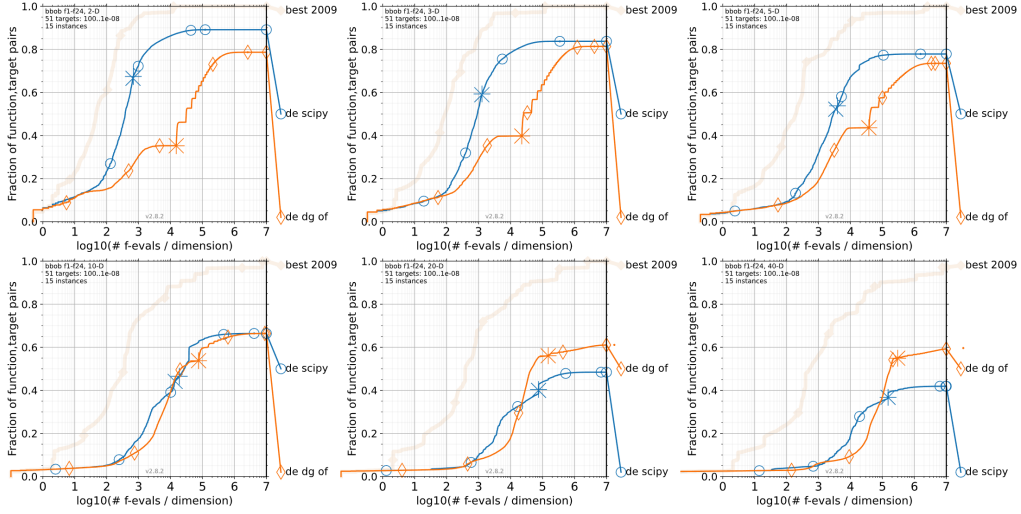


Figure 1: BBOB benchmark results

We can see that our algorithm generally achieves worse results in lower dimensions and takes the lead in higher ones; more on this in the section Analysis of the results.

5.2 Convergence study

We can see that both algorithms converge very similarly and the differences between them are negligible.

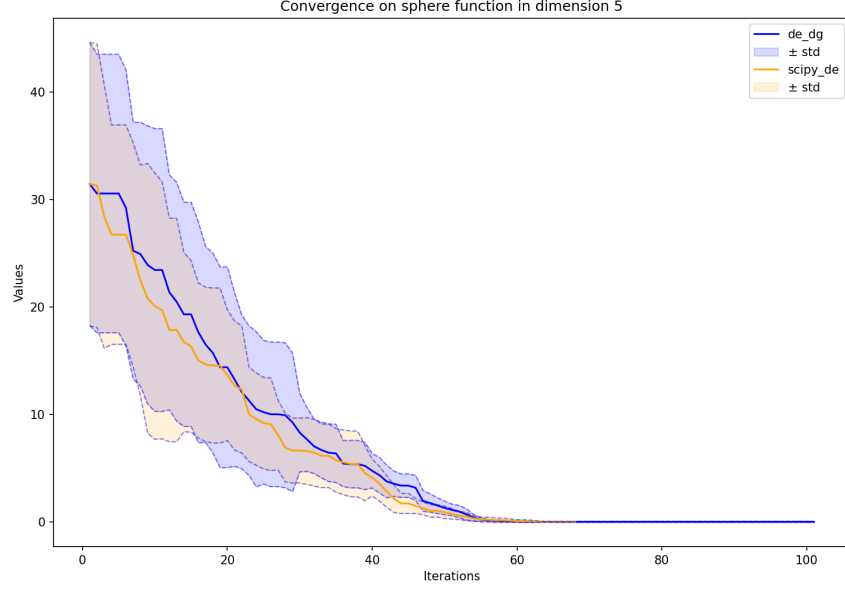


Figure 2: Convergence plot, Values - objective function values

6 Analysis of the results

To analyze the results, we performed the Wilcoxon signed-rank test for paired observations to check whether the differences between the algorithms are statistically significant. We tested the null hypothesis that the distribution of differences between the results of the two algorithms is symmetric around zero (i.e. the algorithms perform the same). The alternative hypothesis assumed that there is some difference between them, without assuming in advance which one is better. One test was performed for each dimension. A pair in the test consisted of the median of Δf over 15 runs of a given function for the SciPy algorithm and the median of Δf over 15 runs of the same function for our algorithm, where:

- $\Delta f = f(x_{best}) - f(x_{opt})$
- x_{best} - the best solution found by the algorithm
- x_{opt} - the optimal solution

Additionally, we applied a cutoff threshold of 10^{-8} : all Δf values below it are treated as 10^{-8} . We did this because in BBOB an error below 10^{-8} is considered a success. In the results table:

- Scipy_Median and DE_DG_Median are the median of the medians of Δf
- Wins - number of wins of the SciPy algorithm, $\Delta f_{scipy} < \Delta f_{de_dg}$
- Ties - $\Delta f_{scipy} = \Delta f_{de_dg}$
- Losses - number of losses of the SciPy algorithm, $\Delta f_{scipy} > \Delta f_{de_dg}$

Dimension	Num_Functions	Scipy_Median	DE_DG_Median	Wins	Ties	Losses	P_Value
2	24	8.30e-07	1.80e+00	20	1	3	0.0001
3	24	1.29e-05	8.15e-01	17	2	5	0.0003
5	24	3.59e-05	5.60e-01	14	3	7	0.0064
10	24	9.60e-04	1.00e-03	9	3	12	0.3392
20	24	9.15e-01	1.25e-01	8	2	14	0.9095
40	24	1.95e+00	1.25e-01	8	2	14	0.9353

Table 1: Wilcoxon test results

For dimensions 2, 3 and 5 the p-value < 0.05 , which allows us to reject the null hypothesis. The difference between the algorithms is statistically significant. Judging by the number of wins, we can conclude that the standard DE/rand/1/bin implementation from the SciPy library performs better in low dimensions.

For higher dimensions the obtained p-value is well above the significance threshold $\alpha = 0.05$. This means there are no grounds to reject the null hypothesis. Statistically, both algorithms show comparable performance. Even though DE/rand/1/bin from SciPy lost more often than it won against our algorithm, we cannot claim that it was worse.

7 Discussion

Unfortunately, the DG heuristic did not live up to expectations and generally performs worse than standard DE. It did not solve the problem of overly fast convergence, and it added extra computation for calculating the diversity, which can be very expensive, so the running time of our algorithm in the BBOB benchmark was considerably longer than that of the SciPy DE implementation.